6. PLANO DE GARANTIA DA QUALIDADE

Padrões de Desenvolvimento

Tema	Padrão adotado
Linguagem e versão	Java 15 ou superior (recomendado JDK 21)
Nomes	Classes em PascalCase (`ColaboradorComissionado`); métodos e variáveis em camelCase (`criarColaborador`); constantes em MAIÚSCULAS com sublinhado
Idioma	Atributos do modelo de dados mantêm a nomenclatura já adotada (`id`, `name`, `baseSalary`); métodos de aplicação e mensagens ao usuário em português
Formatação	Indentação de 4 espaços, uma classe por arquivo e chaves sempre presentes
Estrutura	Cada método com uma única responsabilidade; cálculo e validação separados da leitura do console sempre que possível
Validação	Toda entrada do usuário é validada antes de ser usada em qualquer cálculo (RNF002)
Valores fixos	Sem números ou textos repetidos soltos no código; usar constantes
Comentários	Apenas onde explicam o motivo de uma decisão; sem código comentado ou morto
Arquivos	`data.json`, pastas de compilação (`out`, `target`) e configurações de editor não vão para o repositório (`.gitignore`)

Controle de Versão

- **Ferramenta:** Git, com repositório hospedado no GitHub.
- **Branch principal:** `main`, sempre em estado compilável e funcional.
- **Branches de trabalho:** curtas e nomeadas pelo tipo da mudança, como `feat/relatorio-resumo`, `fix/nome-obrigatorio`, `docs/plano-de-testes` e `test/calculo-salario`.
- **Integração:** toda mudança entra na `main` por Pull Request, com descrição do que foi feito e do requisito atendido.
- **Commits:** pequenos, com uma mudança por commit, no formato `tipo: descrição em português`, citando o requisito quando houver (exemplo: `fix: valida nome obrigatório (RN002)`).

Tipo	Uso
`feat`	Nova funcionalidade
`fix`	Correção de defeito
`docs`	Documentação
`test`	Criação ou alteração de testes
`refactor`	Mudança de estrutura sem alterar o comportamento
`chore`	Configuração, build e manutenção

- **Versões dos documentos:** numeração `maior.menor` (1.0, 1.1, 2.0). Os identificadores de requisitos (RF, RN, RNF), casos de uso (UC) e casos de teste (CT) não são renumerados, para preservar a rastreabilidade.

Revisões Técnicas

Processo

1. O autor abre o Pull Request com a descrição da mudança e o requisito relacionado.
2. Outro integrante (nunca o próprio autor) atua como revisor e aplica o checklist abaixo.
3. O revisor registra o resultado no Pull Request: **aprovado**, **aprovado com ressalvas** ou **reprovado**.
4. Cada não conformidade vira um comentário ou uma Issue, com responsável e prazo.
5. O autor corrige e solicita nova revisão.
6. A mudança só é integrada à `main` após aprovação.

Documentos passam pelo mesmo processo: leitura cruzada entre o texto, os requisitos do cliente e o código.

Checklist de revisão

Item	Pergunta
Compilação	O projeto compila e executa sem erros?
Requisito	A mudança atende ao requisito citado, sem contrariar outro?
Regras de negócio	As validações (RN001 a RN007) continuam funcionando?
Cálculo	As fórmulas de cada categoria permanecem corretas?
Entradas	Entradas inválidas são tratadas sem encerrar o sistema?
Padrões	Nomes, formatação e estrutura seguem os padrões de desenvolvimento?
Testes	Os testes relacionados foram criados ou atualizados e estão passando?
Repositório	Não há arquivos indevidos nem credenciais no commit?
Documentação	Documentos e matriz de rastreabilidade foram atualizados?

Registro: cada revisão formal gera uma ata com projeto, data, participantes, itens revisados, não conformidades, correções e decisão. As atas ficam na pasta `10_Qualidade`.

Critérios de Qualidade

Critério	Descrição	Requisitos	Verificação
Corretude	Salários calculados conforme as fórmulas de cada categoria	RF002 a RF004, RF006, RN008	Testes unitários e de aceitação
Integridade dos dados	Matrícula única e valores válidos	RN001 a RN007	Testes unitários e de sistema
Confiabilidade	Entradas inválidas não derrubam o sistema	RNF002	Teste de sistema
Persistência	Dados mantidos após reiniciar	RNF003	Teste de sistema
Usabilidade	Menu claro, utilizável sem treinamento	RNF001	Teste de aceitação
Manutenibilidade	Novos tipos de colaborador incluídos sem alterar os existentes	RNF004	Inspeção técnica
Rastreabilidade	Todo requisito ligado a caso de uso, classe e caso de teste	Todos	Matriz de rastreabilidade

Métricas Adotadas

Métrica	Meta	Como é medida
Cobertura de testes unitários	80% ou mais	Relatório do JaCoCo (cobertura de linhas)
Defeitos críticos abertos	0	Issues com rótulo de severidade crítica
Requisitos atendidos	100%	Requisitos com caso de teste aprovado ÷ total de requisitos
Casos de teste executados	100%	Executados ÷ planejados
Casos de teste aprovados	100% na execução final	Aprovados ÷ executados
Defeitos corrigidos (críticos e altos)	100%	Corrigidos ÷ encontrados
Pull Requests revisados antes da integração	100%	Pull Requests aprovados ÷ integrados
Requisitos com caso de teste associado	100%	Matriz de rastreabilidade

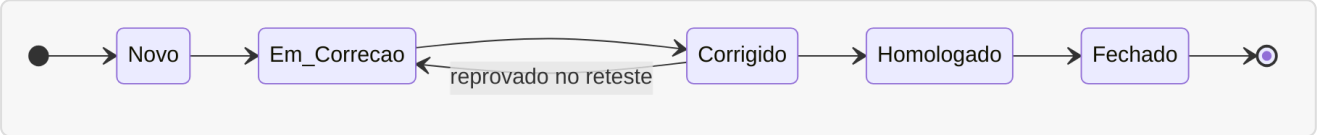
Gestão de Defeitos

Os defeitos são registrados como Issues no GitHub, um por defeito, com os campos: identificação, descrição, passos para reproduzir, severidade, prioridade, status, responsável e solução aplicada.

Severidade	Definição
Crítica	Impede o uso do sistema, corrompe dados ou calcula salário incorretamente
Alta	Funcionalidade principal com falha e sem alternativa
Média	Falha com alternativa de uso
Baixa	Problema de texto, formatação ou aparência

Prioridade	Definição
Alta	Corrigir imediatamente
Média	Corrigir antes do próximo marco do cronograma
Baixa	Corrigir se houver tempo

Ciclo de vida de um defeito:



Critérios de Aprovação

O sistema é aprovado para homologação quando, cumulativamente:

- todos os requisitos funcionais, regras de negócio e requisitos não funcionais estão atendidos, com caso de teste aprovado;
- não há defeitos críticos nem altos abertos;
- a cobertura de testes unitários é de 80% ou mais;
- todos os casos de teste planejados foram executados;
- a documentação está completa e a matriz de rastreabilidade atualizada.

Resultado	Condição
Homologado	Todos os critérios acima cumpridos
Homologado com ressalvas	Critérios cumpridos, com defeitos médios ou baixos abertos e registrados
Não homologado	Algum critério não cumprido